Pattern Matching: Customization Point for Open Sum Types

Document #:	P3521R0
Date:	2024-12-17
Project:	Programming Language C++
Audience:	Evolution Library Evolution

Reply-to:	Michael Park <mcypark@gmail.com> Zach Laine <whatwasthataddress@gmail.com>

Contents

§ 1 Introduction

This is an accompanying paper for [P2688R3] with respect to customization points in the library. At the Wrocław meeting in November 2024, the following poll was taken in EWG:

> Poll: [P2688R3] — Pattern Matching: `match` Expression, we encourage more work on the language-only paper towards C++26 in the next meeting (note: voting against this poll does not exclude getting pattern matching in C++29)

SF	F	N	A	SA
17	16	6	1	9

The "language-only paper" still relies on library facilities, the same way that structured bindings is a "language-only" feature that relies on library facilities as a hook for customization.

A Reflection-based Tuple-like and Variant-like Customization Points has been explored, but the design decision for [P2688R3] targeting C++26 is to use the existing facilities.

This makes it such that even a "language-only paper" is plenty useful without any library changes. However, there is one customization point that is currently missing: the ability for user-defined types to opt-in as an open sum type.

This paper proposes to add that customization point.

§ 2 Motivation and Scope

§ 2.1 Sum Types

First, definitions. A "sum type" `SUM` is a type that represents one of several types in the set `S` = {`T0`, `T1`, … `TN`}. Which `Ti` `SUM` represents is a runtime property.

The set of types `S` is part of the definition of `SUM`, but the exact elements of `S` might not be known at the time `SUM` is defined. For instance: for `std::variant<int, double>`, `S` = `{int, double}`; however, for `std::any`, `S` = `{x | x is a copyable C++ type}`. Though we know that `std::any` can hold any copyable type, we don't know all the copyable types that might exist in a program.

If all the elements `S` are known at the time that `SUM` is defined, we say that `SUM` is a "closed" sum type (e.g. `std::variant<int, double>`). If the members of the set are *not* known at the time `SUM` is defined, we say that `SUM` is an "open" sum type (e.g. `std::any`).

§ 2.2 Implications for Pattern Matching

[P2688R3] proposes an alternative pattern which has syntax like the following:

```
int f(const std::variant<int, double>& v) {
  return v match {
    int: let i => i;
    double: let d => int(d);
  };
}
```

This uses the "variant-like" protocol which uses the existing set of variant helpers: `variant_size`, `variant_alternative`, `get`, and `index`.

However, for open sum types, there is no such existing facility. `std::any` and `std::exception_ptr` are examples of such types.

C++23	P2688 with This Paper

`int f(const std::any& a) {`
`  if (auto* i = std::any_cast<int>(&a)) {`
`  return *i;`
`  } else if (auto* d =`
`  std::any_cast<double>(&a)) {`
`  return *d;`
`  } else {`
`  return -1;`
`  }`
`}`
	`int f(const std::any&`
	`  a) {`
	`  return a match ->`
	`  int {`
	`  int: let i => i;`
	`  double: let d =>`
	`  d;`
	`  _ => -1;`
	`  };`
	`}`

[P2927R2] mentions a desired pattern matching use case for `std::exception_ptr`. The example is written with [P1371R3], but the following is what it would look like with [P2688R3]:

C++23	P2688 with This Paper
`void f(const std::exception_ptr& eptr) {` `  if (auto* e =` `  std::exception_ptr_cast<logic_error>` `  (eptr)) {` `  // use `*e` ` } else if (auto* e =` ` std::exception_ptr_cast<exception>` ` (eptr)) {` ` // use `*e` `  } else if (ep == nullptr) {` `  std::print("no exception");` `  } else {` `  std::print("some other exception");` `  }` `}`	`void f(const` `  std::exception_ptr&` `  eptr) {` `  eptr match {` `  logic_error: let e =>` `  // use `e` ` exception: let e => //` ` use `e` `  nullptr =>` `  std::print("no` `  exception");` `  _ => std::print("some` `  other exception");` `  };` `}`

This paper proposes `try_cast` as the customization point that pattern matching invokes to test and extract the state of open sum types.

§ 3 Design Overview

The Alternative Pattern as proposed in [P2688R3] considers the following 3 cases in order:

1. Variant-like
2. Casts
3. Polymorphic types

This is modeled similarly to structured bindings protocol, which considers the following 3 cases:

1. Array
2. Tuple-like
3. Aggregate types

The cast protocol is the only one that needs library support for `std` entities to benefit from it.

The proposal is to add `try_cast` overloads that handle casts of `std::any` using `std::any_cast`, and casts of `std::exception_ptr` using `std::exception_ptr_cast` from P2927R2.

This works because the language feature itself uses `try_cast` if available. Casts of user-defined types using calls to `try_cast` found via ADL, will also work, such as:

```
namespace ns {
  struct Widget { /* ... */ };

  template <typename T>
  const T* try_cast(const Widget& w) noexcept {
    return // ...
  };
}
```

§ 3.1 How does pattern matching actually use this?

Given an example like:

```
subject match {
  type: subpattern => // ...;
  _ => // ...;
};
```

Conceptually: `try_cast<type>(subject)` is invoked, which returns an optional result that can be tested and dereferenced to match the *subpattern*.

More precisely: Let `E` be `try_cast<type>(subject)`. The call to `try_cast` is done using ADL-only lookup, just as is done with calls to `get` for structured bindings (see 9.6 [dcl.struct.bind]).

The match is well-formed if and only if:

— `E` is well-formed;

— `E` is contextually convertible to `bool`; and

— `E` is dereferencable.

A well-formed match succeeds if and only if `E` contextually converts to `true` and `*E` matches *subpattern*.

§ 4 Proposed Wording

§ 22.7.2 [any.synop] Header <any> synopsis

```
  template<class T>
    T any_cast(const any& operand);
  template<class T>
    T any_cast(any& operand);
  template<class T>
    T any_cast(any&& operand);

  template<class T>
    const T* any_cast(const any* operand) noexcept;
  template<class T>
    T* any_cast(any* operand) noexcept;

+ template<class T, same_as<any> Any>
+   const T* try_cast(const Any& operand) noexcept;
+ template<class T, same_as<any> Any>
+   T* try_cast(Any& operand) noexcept;
+ template<class T, same_as<any> Any>
+   T* try_cast(Any&& operand) noexcept;
```

§ 22.7.5 [any.nonmembers] Non-member functions

```
+ template<class T, same_as<any> Any>
+   const T* try_cast(const Any& operand) noexcept;
+ template<class T, same_as<any> Any>
+   T* try_cast(Any& operand) noexcept;
+ template<class T, same_as<any> Any>
+   T* try_cast(Any&& operand) noexcept;
```

Mandates: `is_void_v<T>` is `false`.

Returns: `any_cast<T>(&operand)`

§ 17.9.2 [exception.syn] Header <exception> synopsis

```
  exception_ptr current_exception() noexcept;
  [[noreturn]] void rethrow_exception(exception_ptr p);
  template <class E>
    const E* exception_ptr_cast(const exception_ptr& p) noexcept;
  template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

```
+ template<class E>
+   const E* try_cast(const exception_ptr& p) noexcept;
```

§ 17.9.7 [propagation] Exception propagation

```
+ template<class E>
+   const E* try_cast(const exception_ptr& p) noexcept;
```

Mandates: `E` is a *cv*-unqualified complete object type. `E` is not an array type. `E` is not a pointer or pointer-to-member type.

Returns: `exception_ptr_cast<E>(p)`

§ 5 Design Alternatives

§ 5.1 Overload `any_cast`

Adding new overloads of `any_cast` was considered as a way to opt-in as an any-like type. Effectively designating variant-like as the closed sum type opt-in, and any-like as the open sum type opt-in.

The reason this approach is not taken is purely technical. The current overloads of `any_cast` include versions that takes `any` by reference. Since `any` is implicitly constructible from anything, testing the validity of `any_cast` is virtually always true.

§ 5.2 Parameters as Pointers

It was considered to take the parameters by pointer instead to avoid the implicit construction of `any` from anything problem:

```
namespace std {
  template <typename T>
  const T* try_cast(const any* a) noexcept { return any_cast<T>(a); }

  template <typename T>
  T* try_cast(any* a) noexcept { return any_cast<T>(a); }

  template <typename T>
  const T* try_cast(const exception_ptr* p) noexcept {
    if (!p) return nullptr;
    return exception_ptr_cast<T>(*p); // P2927R2
  }
}
```

This solves the problem of having to specify `same_as<any>` constraint, but it makes the rest of the overloads less obvious and familiar.

§ 5.3 Use a `std::hash`-like Approach

This approach is to make `try_cast` be a class template that users specialize, rather than an overload set.

```
namespace std {
  template <typename>
  struct try_cast; // undefined

  template <>
  struct try_cast<std::any> {
    template <typename T>
    static const T* to(const std::any& a) noexcept {
      return std::any_cast<T>(&a);
    }

    template <typename T>
    static T* to(std::any& a) noexcept {
      return std::any_cast<T>(&a);
    }
  };

  template <>
  struct try_cast<std::exception_ptr> {
    template <typename T>
    static const T* to(const std::exception_ptr& p) noexcept {
      return std::exception_ptr_cast<T>(p); // P2927R2
    }
  };
}
```

This allows the parameters to become references again. There are two downsides of this approach:

1. The user-defined type needs to re-open the `std` namespace in order to provide the specialization.
2. The API becomes `std::try_cast<From>::to<To>(from)`, compared to the proposed API which would be simply `try_cast<To>(from)`.

§ 5.4 Create a `try_cast` CPO

This approach was seriously considered by the authors, but discarded. We wrote wording for it and everything. While it may be useful for users in general, it is a terrible fit for use by a language feature. The CPO would have to live in a particular header. That header would then need to be included wherever a `type:` pattern was applied to an open sum type.

In the general case, the user would have no obvious way of noticing that this requirement was not met. They would just get a diagnostic telling them to `#include` the right header. Users already have to deal with this occasionally when the compiler informs them that they forgot to include `<initializer_list>`.

Note that the tuple protocol uses direct calls to the overloads that it requires for producing structured bindings, and this decision is in line with that. In any case, the decision not to use a CPO is already made; it's part of the language feature.

§ 6 References

[P1371R3] Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. 2020-09-15. Pattern Matching.

https://wg21.link/p1371r3

[P2688R3] Michael Park. 2024-10-16. Pattern Matching: `match` Expression.

https://wg21.link/p2688r3

[P2927R2] Gor Nishanov, Arthur O'Dwyer. 2024-04-16. Observing exceptions stored in exception_ptr.

https://wg21.link/p2927r2